Solution: Index Pairs of a String

Problem Statement

Given a string `text` and a list of strings `words`, identify all `[i, j]` index pairs such that the substring `text[i...j]` is in words.

These index pairs should be returned in ascending order, first by the start index, then by the end index. Find every occurrence of each word within the `text`, ensuring that overlapping occurrences are also identified.

Examples

1. - **Input**: text = `"bluebirdskyscraper"`, words = `["blue", "bird", "sky"]`
   - **Expected Output**: `[[0, 3], [4, 7], [8, 10]]`
   - **Justification**: The word "blue" is found from index 0 to 3, "bird" from 4 to 7, and "sky" from 8 to 10 in the string.
2. - **Input**: text = `"programmingisfun"`, words = `["pro", "is", "fun", "gram"]`
   - **Expected Output**: `[[0, 2], [3, 6], [11, 12], [13, 15]]`
   - **Justification**: "pro" is found from 0 to 2, "gram" from 3 to 6, "is" from 11 to 12, and "fun" from 13 to 15.
3. - **Input**: text = `"interstellar"`, words = `["stellar", "star", "inter"]`
   - **Expected Output**: `[[0, 4], [5, 11]]`
   - **Justification**: "inter" is found from 0 to 4, and "stellar" from 5 to 11. "star" is not found.

Solution

To solve this problem, we will use a trie data structure, which is particularly efficient for managing a set of strings and performing quick searches for patterns within a text. A trie, also known as a prefix tree, allows us to efficiently store and retrieve strings with a common prefix, making it ideal for our purpose of identifying substrings within a given text.

The approach involves two main phases: building the trie with the given list of words and then using it to find the index pairs in the text. The trie's structure enables us to search for each word in the text in an optimized manner, significantly reducing the number of comparisons needed compared to brute force methods.

Step-by-Step Algorithm

1. **Initialize the Trie**:
   - Create a trie and insert all the words from the given list into it. This is done in the first part of the code.
2. **Search and Record Index Pairs**:
   - Iterate over each character in the text. This character will act as the starting point for potential matches.
   - For each starting character, initiate a pointer `p` at the root of the trie.
   - Then, iterate over the text starting from the current starting point until the end of the text. In each iteration:
     - Check if the current character exists as a child of the current trie node pointed by `p`.
     - If it does not exist, break out of the inner loop as no further matching is possible from this starting point.
     - If it exists, move the pointer `p` to this child node.
     - Check if the current node `p` is a leaf node (indicated by `p.isEnd` being `true`). If it is, it means a complete word from the list has been found.
     - Record the start and end indices of this word. The start index is the position of the starting character, and the end index is the current position in the text.
   - Continue this process for each character in the text to ensure all occurrences, including overlapping ones, are found.

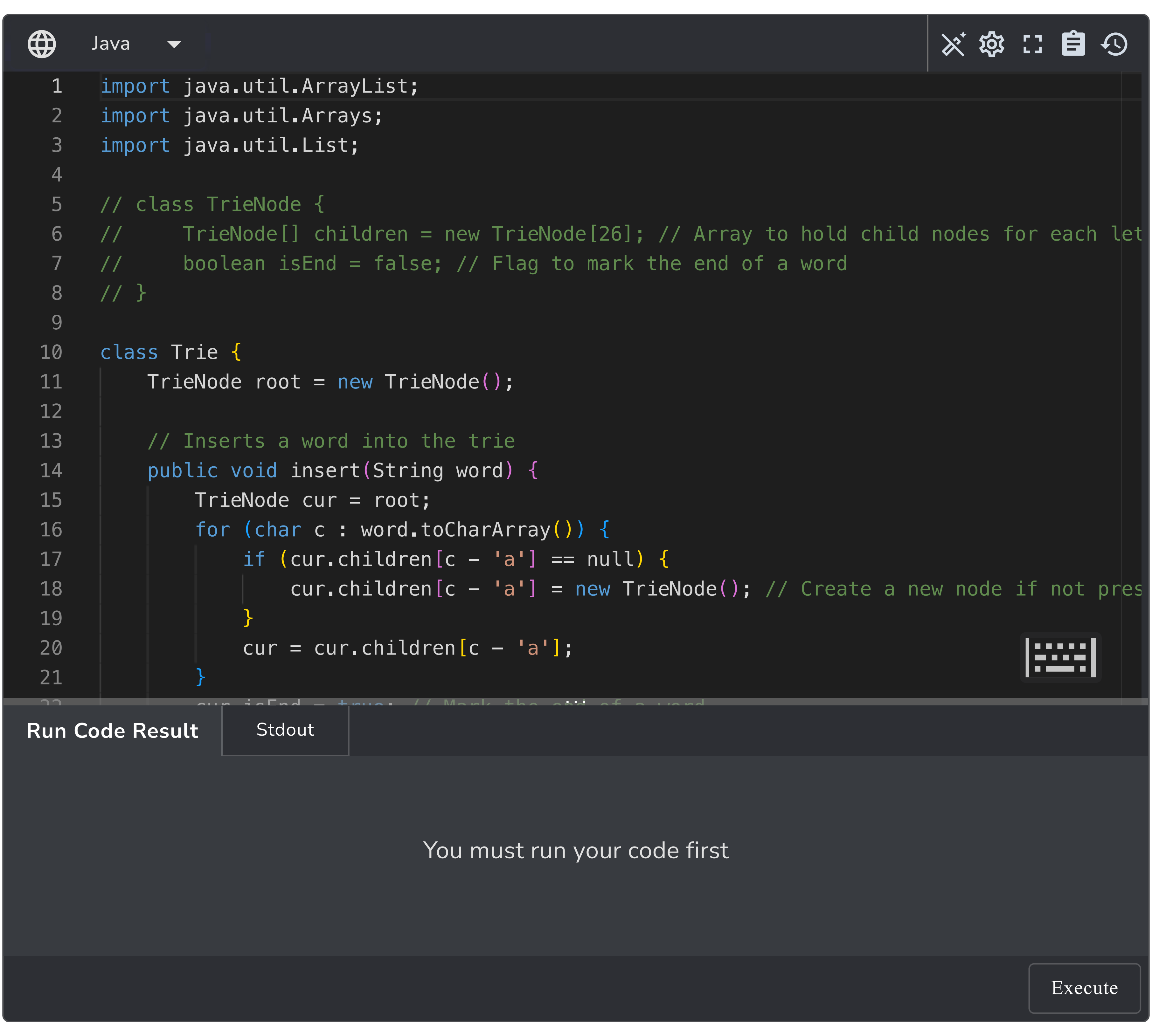
Algorithm Walkthrough

Using the input text = `"programmingisfun"`, words = `["pro", "is", "fun", "gram"]`:

1. **Initialize the Trie with Words**:
   - "pro", "is", "fun", and "gram" are inserted into the trie.
2. **Search and Record Index Pairs**:
   - Start at index 0 (`'p'` in `"programmingisfun"`):
     - Pointer `p` is at the root. Finds 'p', moves to 'r', then 'o'. `p.isEnd` is `true` at 'o', so record [0, 2].
   - At index 1 (`'r'`), no match is found.
   - similarly, At index 2 (`'o'`), no match is found.
   - At index 3 (`'g'`):
     - Finds 'g', 'r', 'a', 'm'. `p.isEnd` is `true` at 'm', so record [3, 6].
   - At index 11 (`'i'`):
     - Finds 'i', 's'. `p.isEnd` is `true` at 's', so record [11, 12].
   - At index 13 (`'s'`):
     - Finds 'f', 'u', 'n'. `p.isEnd` is `true` at 'n', so record [13, 15].
3. **Compile Results**:
   - The results [[0, 2], [3, 6], [11, 12], [13, 15]] are compiled into the 2D array `ans` and returned.

By following these steps, the algorithm efficiently locates all occurrences of the words in the text, including overlapping ones, using the trie structure for optimized searching.

Code



```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

// class TrieNode {
//     TrieNode[] children = new TrieNode[26]; // Array to hold child nodes for each let
//     boolean isEnd = false; // Flag to mark the end of a word
// }

class Trie {
    TrieNode root = new TrieNode();

    // Inserts a word into the trie
    public void insert(String word) {
        TrieNode cur = root;
        for (char c : word.toCharArray()) {
            if (cur.children[c - 'a'] == null) {
                cur.children[c - 'a'] = new TrieNode(); // Create a new node if not pres
            }
            cur = cur.children[c - 'a'];
        }
```

You must run your code first

Time and Space Complexity Analysis

Time Complexity:

- **Building the Trie**: O(N * L), where N is the number of words and L is the average length of these words.
- **Finding Index Pairs**: O(T^2 * L), where T is the length of the text string and L is the length of the longest word.
- **Overall**: O(N * L + T^2 * L)

Space Complexity:

- **Trie Storage**: O(N * L), for storing N words each of average length L.
- **Overall**: O(N * L), dominated by the Trie storage.

← Previous
Index Pairs Of A String (Easy)
Next →
Design Add And Search Words Data Structure (Medium)
☐ Mark as Completed